Week-3 skill

Session-5

Task-1

TO Apply Single Quotes, Preserve Literal Content, Ignore Variable Expansion, Store Quoted Strings, Validate Parsing Results, Test Edge Cases

kiran@k: ~/OSSP

```
 GNU nano 8.7.1
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 500

void parseSingleQuotes(char *input, char *output) {
    int i = 0;
    int j = 0;
    int inSingleQuote = 0;

    while (input[i] != '\0') {

        if (input[i] == '\'') {
            inSingleQuote = !inSingleQuote;
            i++;
            continue;
        }

        output[j++] = input[i];
        i++;
    }

    output[j] = '\0';
}

int validateQuotes(char *input) {
    int singleQuoteCount = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '\'') {
            singleQuoteCount++;
        }
    }

    if (singleQuoteCount % 2 != 0) {
        return 0;
    }

    return 1;
}

int main() {
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("================================\n");
    printf("       SINGLE QUOTE PARSER\n");
    printf("================================\n");

    printf("\nEnter a command: ");
    fgets(input, MAX_INPUT, stdin);

    input[strcspn(input, "\n")] = '\0';

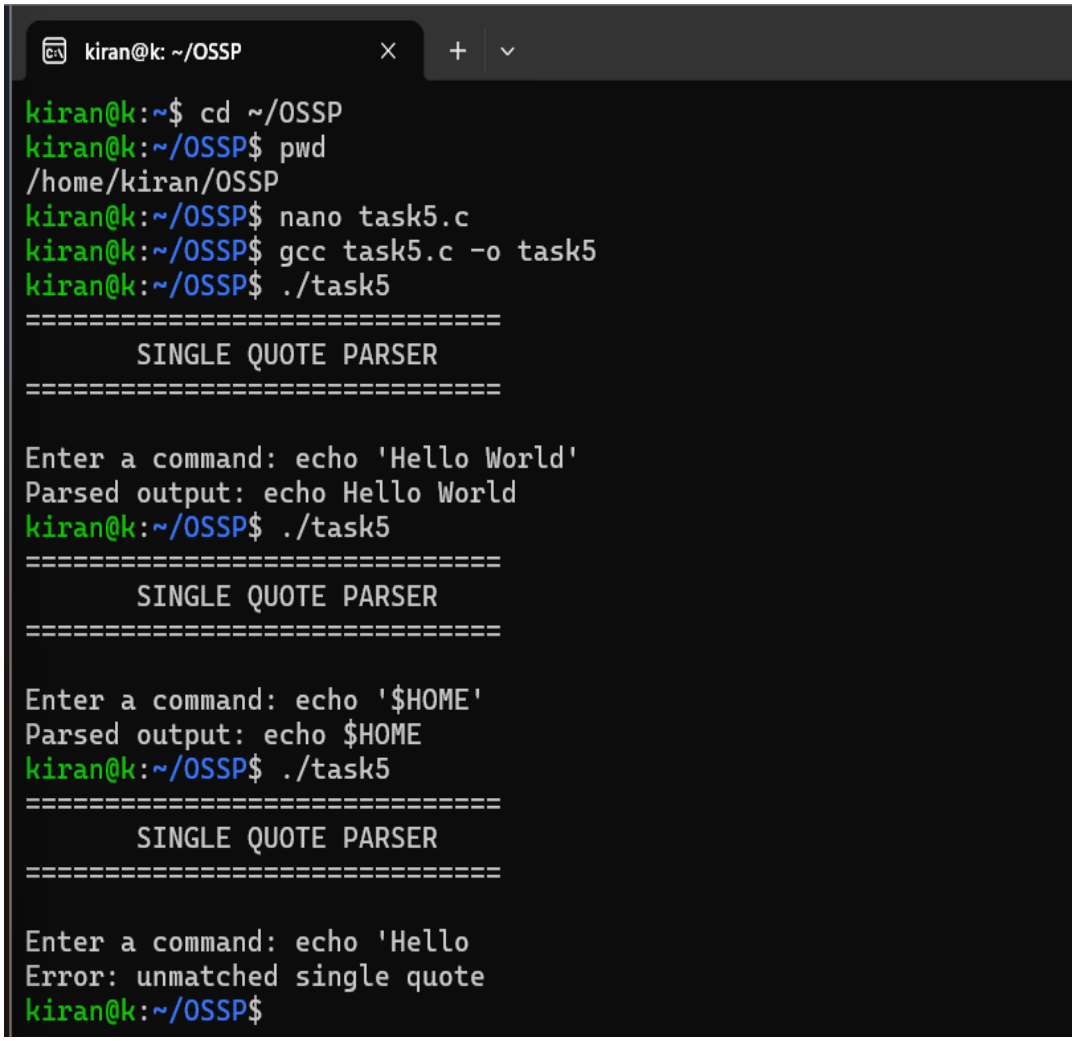
    if (!validateQuotes(input)) {
        printf("Error: unmatched single quote\n");
        return 1;
    }

    parseSingleQuotes(input, output);

    printf("Parsed output: %s\n", output);

    return 0;
}
```

OUTPUT:



Report: In this task, I implemented handling of single quotes in a shell command using C. The text inside single quotes was treated as literal content, so spaces and special characters were preserved. Variable names such as $HOME were not expanded. The program also checked whether the single quotes were properly closed and displayed an error for unmatched quotes. Different inputs were tested to verify the parsing results.

Task-2

To Apply Double Quotes, Preserve Spaces, Allow Variable Expansion, Parse Nested Tokens, Validate Outputs, Test Quoted Commands

kiran@k: ~/OSSP

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 500

void expandVariable(const char *var, char *output, int *j) {
    char *value = getenv(var);

    if (value != NULL) {
        strcpy(&output[*j], value);
        *j += strlen(value);
    }
}

void parseDoubleQuotes(char *input, char *output) {
    int i = 0;
    int j = 0;
    int inDoubleQuote = 0;

    while (input[i] != '\0') {

        if (input[i] == '"') {
            inDoubleQuote = !inDoubleQuote;
            i++;
            continue;
        }

        if (input[i] == '$' && inDoubleQuote) {
            char var[100];
            int k = 0;

            i++;

            while (input[i] != '\0' &&
                   (input[i] == '_' ||
                   (input[i] >= 'A' && input[i] <= 'Z') ||
                   (input[i] >= 'a' && input[i] <= 'z') ||
                   (input[i] >= '0' && input[i] <= '9')) &&
                   k < 99) {

                var[k++] = input[i++];
            }

            var[k] = '\0';

            expandVariable(var, output, &j);
            continue;
        }

        output[j++] = input[i++];
    }

    output[j] = '\0';
}

int validateQuotes(char *input) {
    int count = 0;

    for (int i = 0; input[i] != '\0'; i++) {
        if (input[i] == '"') {
            count++;
        }
    }

    return count % 2 == 0;
}

int main() {
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("==============================\n");
    printf("       DOUBLE QUOTE PARSER\n");
    printf("==============================\n");

    printf("\nEnter a command: ");
    fgets(input, MAX_INPUT, stdin);

    input[strcspn(input, "\n")] = '\0';

    if (!validateQuotes(input)) {
        printf("Error: unmatched double quote\n");
        return 1;
    }

    parseDoubleQuotes(input, output);

    printf("Parsed output: %s\n", output);

    return 0;
```

OUTPUT:

```
kiran@k:~/OSSP$ nano task6.c
kiran@k:~/OSSP$
kiran@k:~/OSSP$ gcc task6.c -o task6
kiran@k:~/OSSP$ ./task6
==============================
       DOUBLE QUOTE PARSER
==============================

Enter a command: echo "Hello World"
Parsed output: echo Hello World
kiran@k:~/OSSP$ ./task6
==============================
       DOUBLE QUOTE PARSER
==============================

Enter a command: echo "Hello $USER"
Parsed output: echo Hello kiran
kiran@k:~/OSSP$ ./task6
==============================
       DOUBLE QUOTE PARSER
==============================

Enter a command: echo "Home is $HOME"
Parsed output: echo Home is /home/kiran
kiran@k:~/OSSP$
```

Report: In this task, I implemented double quote handling in C. Double quotes were used to preserve spaces within a command while allowing environment variables such as $USER and $HOME to be expanded. The program checked for unmatched double quotes and displayed the parsed output. Different quoted commands were tested to verify that spaces and variables were handled correctly.

Session-6

Task-1

To Create Process Escape Sequences, Handle Escaped Spaces, Escape Special Symbols, Preserve Characters, Validate Parser Output, Test Complex Inputs

kiran@k: ~/OSSP

GNU nano 8.7.1

```c
#include <stdio.h>
#include <string.h>

#define MAX_INPUT 500
#define MAX_OUTPUT 500

void parseEscapes(char *input, char *output) {
    int i = 0;
    int j = 0;

    while (input[i] != '\0') {

        if (input[i] == '\\') {
            i++;

            if (input[i] != '\0') {
                output[j++] = input[i];
                i++;
            }
        } else {
            output[j++] = input[i];
            i++;
        }
    }

    output[j] = '\0';
}

int main() {
    char input[MAX_INPUT];
    char output[MAX_OUTPUT];

    printf("================================\n");
    printf("       ESCAPE SEQUENCE PARSER\n");
    printf("================================\n");

    printf("\nEnter a command:\n");
    printf("shell> ");

    fgets(input, MAX_INPUT, stdin);

    input[strcspn(input, "\n")] = '\0';

    parseEscapes(input, output);

    printf("\nOriginal Input : %s\n", input);
    printf("Parsed Output  : %s\n", output);

    return 0;
}
```

OUTPUT:

```
kiran@k:~/OSSP$ cd ~/OSSP
nano task7.c
kiran@k:~/OSSP$ gcc task7.c -o task7
kiran@k:~/OSSP$ ./task7
================================
       ESCAPE SEQUENCE PARSER
================================

Enter a command:
shell> hello\ world

Original Input : hello\ world
Parsed Output  : hello world
kiran@k:~/OSSP$ gcc task7.c -o task7
kiran@k:~/OSSP$ ./task7
================================
       ESCAPE SEQUENCE PARSER
================================

Enter a command:
shell> hello\@world

Original Input : hello\@world
Parsed Output  : hello@world
kiran@k:~/OSSP$ ./task7
================================
       ESCAPE SEQUENCE PARSER
================================

Enter a command:
shell> hello\ world\ test

Original Input : hello\ world\ test
Parsed Output  : hello world test
kiran@k:~/OSSP$
```

Report: In this task, I implemented escape sequence handling using the backslash character. Escaped spaces and special characters were processed so that they were treated as normal characters. The backslash was removed during parsing while the escaped character was preserved. Different inputs containing spaces, special symbols, and variables were tested to check the parser output.

Task-2:To Create Child Processes, Execute Programs, Handle Execution Errors, Pass Arguments, Manage Parent Process, Test Command Launching

kiran@k: ~/OSSP

GNU nano 8.7.1

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    printf("================================\n");
    printf("      PROCESS EXECUTION DEMO\n");
    printf("================================\n");

    pid = fork();

    if (pid < 0) {
        perror("fork failed");
        return 1;
    }

    if (pid == 0) {
        printf("\nChild Process:\n");
        printf("PID: %d\n", getpid());
        printf("Executing ls command...\n");

        execlp("ls", "ls", "-l", NULL);

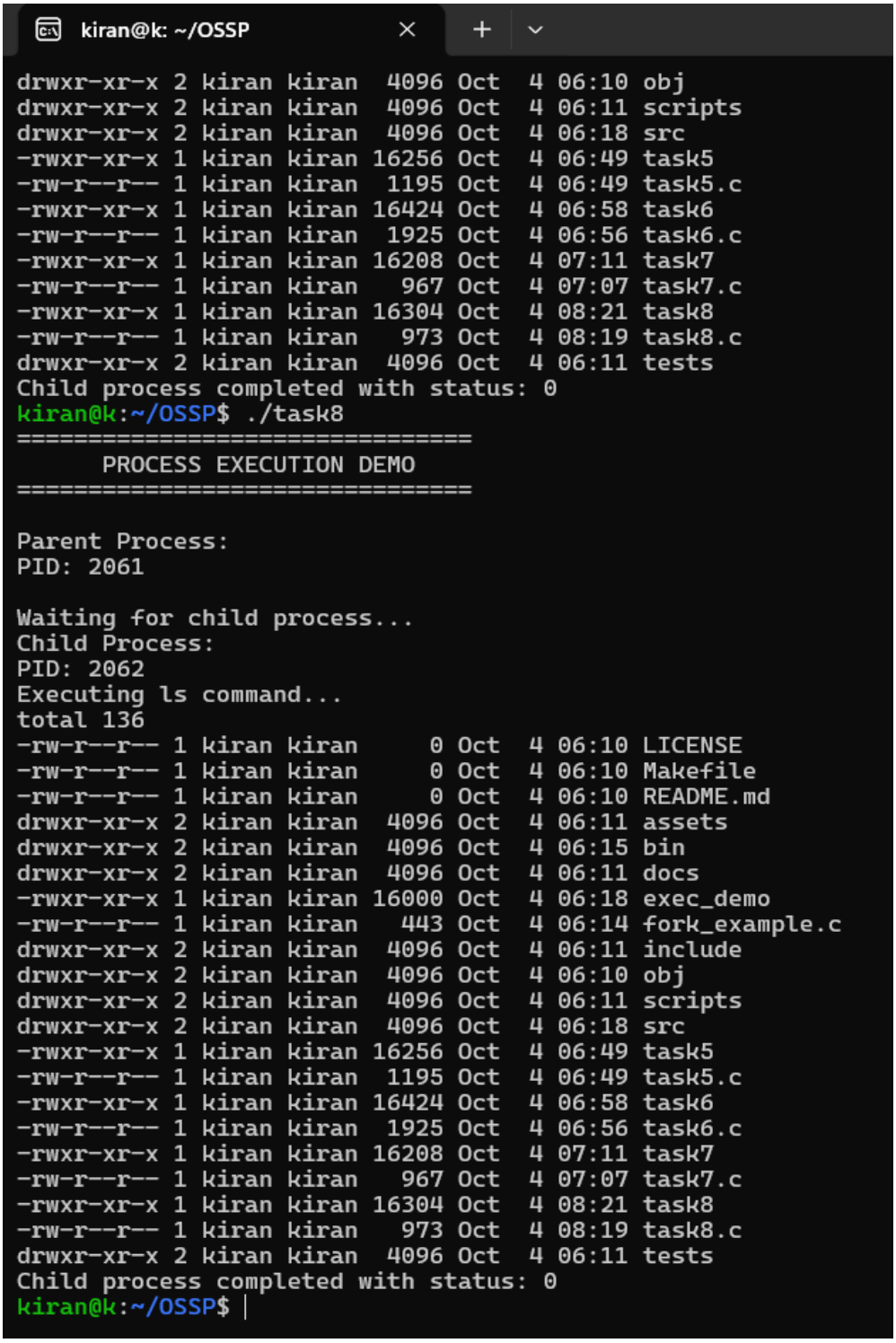
        perror("execlp failed");
        exit(1);
    } else {
        printf("\nParent Process:\n");
        printf("PID: %d\n", getpid());
        printf("Waiting for child process...\n");

        waitpid(pid, &status, 0);

        if (WIFEXITED(status)) {
            printf("Child process completed with status: %d\n",
                   WEXITSTATUS(status));
        }
    }

    return 0;
}
```

OUTPUT:



Report: In this task, I implemented process creation and program execution using fork(), exec() and waitpid(). The parent process created a child process, and the child executed the ls command with arguments using execlp(). The parent waited for the child process to finish and checked its exit status. Error handling was also implemented for cases where the command could not be executed. The program successfully demonstrated process creation, execution, argument passing, and parentchild process management.